ADEPT: Adaptive Defense via Ensemble and Purification Transformer

PIC2 - Master in Computer Science and Engineering
Instituto Superior Técnico, Universidade de Lisboa

Guilherme Silvério de Carvalho Romeiro Leitão — 99951*
guilhermeleitao0202@tecnico.ulisboa.pt

Advisor(s): Fernando Ramos

Abstract The integration of Machine Learning (ML) into Network Intrusion Detection Systems (NIDS) has significantly enhanced the ability to identify complex threats. However, this advancement has introduced a critical vulnerability: the susceptibility to adversarial evasion, where attackers craft subtle perturbations to bypass detection. While the offensive landscape has evolved rapidly, defensive mechanisms have lagged, often relying on unrealistic assumptions—such as complete attacker knowledge—or failing to account for the rigid constraints of network protocols, resulting in "robustified" defensive models trained on traffic that cannot exist in real-world deployments.

To bridge the gap between theoretical security and practical deployability, this thesis introduces ADEPT, a dynamic framework for robust NIDS defense. Unlike static solutions, our approach leverages a heterogeneous stacking strategy that combines diverse architectural paradigms to disrupt gradient-based attack estimation. Furthermore, we implement a generative sanitization module based on the Transformer architecture, which reconstructs suspicious inputs to align with legitimate traffic manifolds before classification. This defense is continuously optimized through a Reinforcement Learning loop that adapts to evolving threats generated by TANTGen, a realistic, black-box traffic generator. Our goal is to protect NIDS against a wide spectrum of adversarial strategies while ensuring the practical deployability of the model.

Keywords — Network Intrusion Detection, Adversarial Defense, Transformer, Reinforcement Learning, Ensemble Learning, Stacking

*I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa (https://nape.tecnico.ulisboa.pt/en/apoio-ao-estudante/documentos-importantes/regulamentos-da-universidade-de-lisboa/).

Contents

1 Introduction **3**
- 1.1 Work Objectives . . . 3
- 1.2 Expected Contributions . . . 4

2 Background **4**
- 2.1 Machine Learning for NIDS . . . 4
  - 2.1.1 Feature Engineering . . . 5
  - 2.1.2 Model Training and Architectures . . . 6
- 2.2 Landscape of Adversarial Machine Learning . . . 7
  - 2.2.1 Brief Overview of Attacks . . . 7
  - 2.2.2 Defenses . . . 7
- 2.3 ML-based Networks . . . 8
  - 2.3.1 Granularity of Analysis . . . 8
  - 2.3.2 Generic ML Applications . . . 8
  - 2.3.3 Network-Specific Challenges . . . 9
- 2.4 Use of Ensemble Models . . . 9
  - 2.4.1 Ensemble Approaches . . . 9
  - 2.4.2 Ensembles for Robustness . . . 9

3 Related Work **10**
- 3.1 Adversarial Defenses in ML-based NIDS . . . 10
  - 3.1.1 Training-time Defenses . . . 10
  - 3.1.2 Inference-time Defenses . . . 12
- 3.2 Reinforcement Learning for Continuous Adaptation . . . 13
- 3.3 Summary . . . 13

4 Solution **13**
- 4.1 Threat Model . . . 13
- 4.2 ADEPT Architecture Overview . . . 15
  - 4.2.1 Rationale for Model Selection . . . 15
  - 4.2.2 Dataset Creation . . . 17
  - 4.2.3 Preprocessing and Feature Engineering . . . 17
  - 4.2.4 Defense Application Loop in Real-life . . . 17
- 4.3 Tools and Frameworks . . . 17

5 Evaluation **18**
- 5.1 Research Questions . . . 18
- 5.2 Experimental Setup . . . 18
  - 5.2.1 Datasets and Threat Generation . . . 18
  - 5.2.2 Implementation Environment . . . 18
- 5.3 Metrics . . . 19

6 Work Schedule **19**

7 Conclusion **19**

Bibliography **21**

1 Introduction

The proliferation of Machine Learning (ML) applications in cybersecurity has fundamentally transformed how organizations defend their networks. Network Intrusion Detection Systems (NIDS) based on ML techniques have demonstrated superior performance over traditional signature-based approaches, achieving great success on benchmark datasets and showing remarkable capability in identifying novel attack patterns [1, 2]. Unlike static rule-based systems, ML models can learn from vast amounts of historical traffic data to distinguish between benign and malicious communications, serving as critical components of modern network security infrastructure.

However, this success has not come without challenges. As ML models become ubiquitous, they also become targets. Adversarial examples—inputs that have been subtly modified to fool ML models while remaining imperceptible to the monitoring systems—represent a fundamental threat to the reliability and trustworthiness of ML-based NIDS [3]. Research has demonstrated that carefully crafted perturbations to network traffic features can cause state-of-the-art classifiers to misclassify malicious flows as benign, potentially allowing attackers to bypass detection entirely [4, 5].

A key distinction in the network domain compared to domains such as computer vision is the complexity of maintaining attack realizability. Network attacks must comply with protocol semantics and respect packet structure constraints while maintaining functional maliciousness. This problem significantly constrains the space of feasible adversarial perturbations and adds an additional layer of complexity to both attack and defense strategies [6], since defenses often require feasible attacks to be effectively evaluated.

While recent research has focused extensively on generating adversarial attacks against NIDS, the development of practical, deployable defenses remains relatively underdeveloped, to the best of our knowledge. Most existing defense mechanisms either operate under unrealistic threat assumptions (such as attacker white-box access) [7, 8] or fail to consider the full lifecycle of adversarial robustness as threats evolve and new attack techniques emerge [9].

To bridge this critical gap, this thesis proposes ADEPT (high-level overview in Figure 1), an adaptive defense framework designed to move beyond static robustification and enable continuous improvement. The rationale for our architecture relies on the use of **ensemble learning**, as combining diverse model architectures has been proven to create "rugged" decision boundaries that significantly mitigate the transferability of adversarial attacks [10, 11]. However, for a defense to be truly adaptive, it must be trained against realistic threats. To this end, we integrate **TANTGen** [12], a novel Transformer-based framework to generate protocol-compliant adversarial network traffic. In our solution, TANTGen serves as a dynamic "Red Team" agent; it continuously generates valid, black-box evasion attempts that adhere to network constraints, providing the necessary feedback for our Reinforcement Learning component to iteratively refine and update the defense strategy in real-time.

1.1 Work Objectives

This thesis aims to design and implement **ADEPT**, a novel adversarial defense system for ML-based NIDS that incorporates the following core objectives:

1. **Feature Engineering and Dynamic Analysis**: Determine good feature representations for adversarial defense.

2. **Ensemble-based Defense**: Explore how ensemble learning methods can be leveraged to increase adversarial robustness by combining diverse model architectures and decision mechanisms.

3. **Input Reconstruction**: Leverage generative models to reconstruct adversarially perturbed packets, projecting them back onto their legitimate form. This approach aims to neutralize adversarial perturbations by effectively "denoising" inputs before being processed by the detection model being protected.

4. **Continuous Defense via Reinforcement Learning**: Develop a reinforcement learning framework that enables the defense system to continuously improve by incorporating feedback from adversarial examples generated by TANTGen [12].

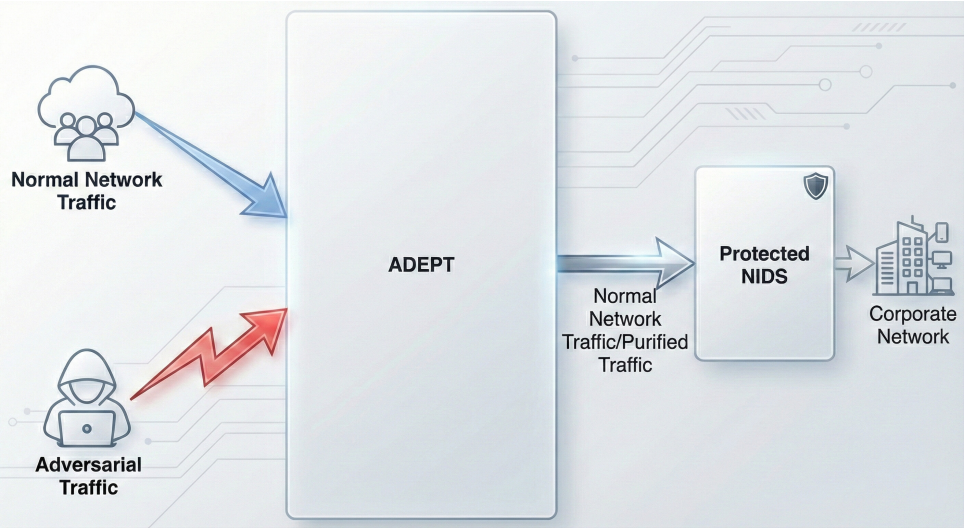


Figure 1: System Overview

5. **Comprehensive Evaluation**: Establish rigorous evaluation testing that assesses defense effectiveness against both known attacks and generated adversarial examples. Here we also expect to conduct individual model prediction segregation analysis to understand how different models within the ensemble contribute to overall robustness—the idea being to identify which architectures are more resilient to specific attack vectors.

1.2 Expected Contributions

This work is expected to deliver an **Adaptive Defense Framework** with **RL-based Continuous Improvement**: a practical, black-box adversarial defense system that operates under realistic threat assumptions and maintains high detection accuracy while resisting evasion attacks. This is coupled with a reinforcement-learning loop that enables the defense to learn from adversarial examples generated in real time, creating a self-improving security system capable of adapting to dynamic threats.

2 Background

The following sections provide essential background on Machine Learning in networking, the landscape of Adversarial Machine Learning (AML), ML-based network applications, and the use of ensemble models for robustness. We cover foundational concepts, challenges, and state-of-the-art techniques relevant to our work.

2.1 Machine Learning for NIDS

ML approaches for Network Intrusion Detection Systems are generally categorized by how they process data and the type of supervision they require. Current literature identifies four primary paradigms used: Supervised, Unsupervised, Semi-Supervised, and Reinforcement Learning [13, 14].

Supervised Learning (Figure 2) is the most prevalent approach in NIDS. It utilizes labeled datasets (where traffic is pre-marked as "benign" or "malicious", for example) to train classifiers. Algorithms such as Random Forest, Support Vector Machines (SVM), and Deep Neural Networks (DNN) learn a mapping from input features to these labels, serving as the standard for detecting known threats [2]. Basically, the model learns to recognize the "fingerprint" of known attacks, which is good for identifying what we already know but is ineffective against new attacks.

Unsupervised Learning (Figure 3) operates without labeled attack samples. Instead, it models the normal behavior of the network (the "baseline") and flags any significant deviation as an anomaly. This capability is crucial for detecting zero-day attacks (previously unknown threats) that supervised models might miss because they haven't seen them before [13, 15].

Semi-Supervised Learning bridges these paradigms by leveraging small labeled sets alongside vast unlabeled data. This is particularly useful in networking, where labeling traffic is expensive and time-consuming. Generative models are often used here to improve decision boundaries by learning the underlying distribution of the traffic [16].

Finally, **Reinforcement Learning (RL)** (Figure 4) introduces the concept of an "agent" that learns by interacting with an environment. In the context of NIDS, an RL agent can dynamically adapt its defense policy (e.g., blocking rules) based on rewards received for successful detections or penalties for false alarms [17, 18].

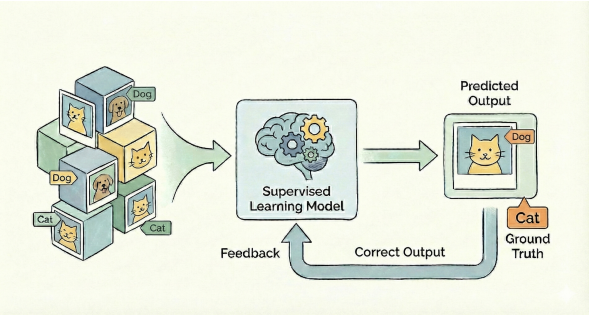


Figure 2: Supervised Learning

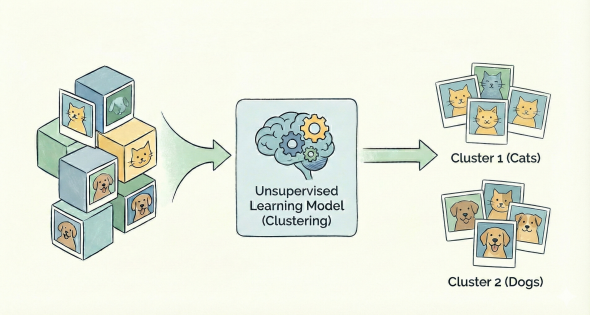


Figure 3: Unsupervised Learning

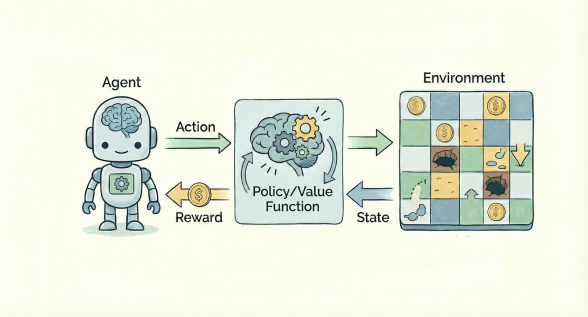


Figure 4: Reinforcement Learning

2.1.1 Feature Engineering

Feature engineering is the process of transforming raw network traffic into a numerical format (vectors) that ML algorithms can process. The quality of these features directly impacts the model's accuracy and computational efficiency [19]. It's essentially about translating the network traffic flows into a structure that the model can better understand and learn from. Key considerations include:

- **Static vs. Dynamic Features:** Static features are extracted from individual packet headers (e.g., source IP, destination port, TCP flags). They are computationally cheap to extract, making them suitable for high-speed networks, but they may lack the context of the communication. Dynamic features (or flow features) aggregate information over a sequence of packets or a time window (e.g., average packet size, inter-arrival times, flow duration). These provide a deeper behavioral view of the traffic, essential for identifying complex patterns [19], but come with higher latency costs.

- **Preprocessing:** Before training, raw features must be normalized. Techniques like Min-Max scaling or Z-score standardization ensure that features with large ranges (like "Flow Duration") do not dominate features with smaller ranges (like "Packet Count") [20]. Additionally, categorical data (like protocol types TCP/UDP) is converted into numerical values using One-Hot Encoding or Label Encoding.
- **Dimensionality Reduction:** Network datasets often contain hundreds of features, many of which are redundant. Dimensionality reduction techniques, such as Principal Component Analysis (PCA) or even AutoEncoders, are used to compress this data into a lower-dimensional space. This reduces the computational load and helps the model focus on the most relevant information [15].

2.1.2 Model Training and Architectures

The training phase involves optimizing the model's parameters to minimize classification error. Recent advancements in NIDS have moved from traditional algorithms to complex deep learning architectures [14]. **Ensemble Methods**, such as Random Forest (Bagging) and XGBoost (Boosting), are widely used through combining multiple "weak" models to create a strong predictor. As highlighted in recent surveys [10], ensembles offer a good balance between training speed and detection accuracy, often outperforming single complex models on tabular network data.

Deep Learning (DL) models are increasingly adopted for their ability to automatically learn feature representations, reducing the reliance on manual feature engineering. Among these, **Convolutional Neural Networks (CNNs)** have proven particularly effective, specifically in their one-dimensional (1D-CNN) configuration, as represented in Figure 5. Unlike their 2D counterparts used in image processing, 1D-CNNs are designed to process sequential data, making them ideal for analyzing network traffic flows or packet payloads where the order of bytes and temporal arrival patterns contains critical information [21]. By sliding convolutional kernels along the temporal dimension of the traffic sequence, these models can extract local patterns—such as specific protocol headers or signature fragments—and combine them into high-level semantic features for classification [22].

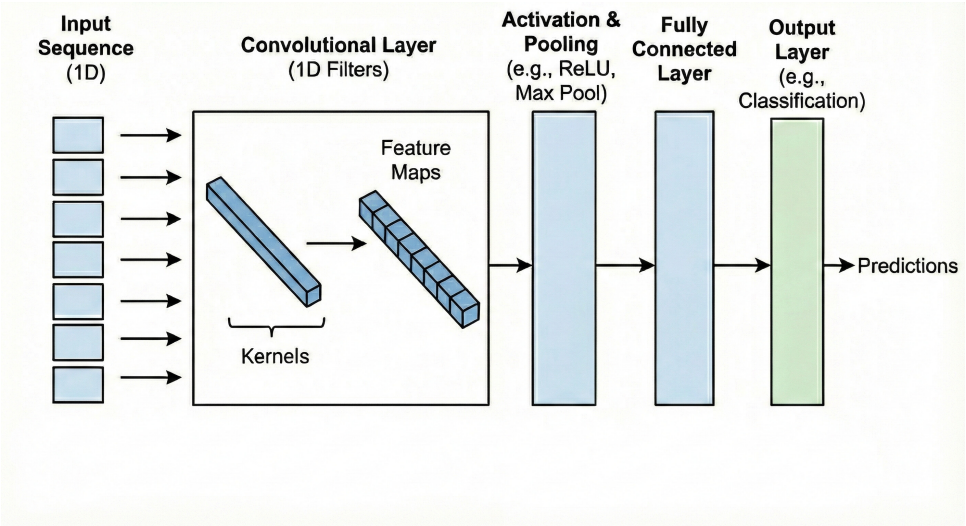


Figure 5: 1D-CNN Architecture Overview

In particular, Transformers [23] utilize self-attention mechanisms to model long-range dependencies in packet sequences, making them highly effective for analyzing encrypted traffic where payload inspection is near impossible [24, 25].

2.2 Landscape of Adversarial Machine Learning

While the methods described above make NIDS effective in normal settings, they can degrade significantly in a harsher adversarial environment where attacks are particularly surreptitious, namely by exploring adversarial example techniques. An adversarial example consists of an input sample that has been subtly modified to cause a machine learning model to misclassify it [26]. AML studies how these models can be manipulated and tricked and how to defend them. In the context of computer networks, and specifically in the attack domain, this challenge is unique due to the ***Inverse Feature Mapping (IFM) problem***: unlike in image classification where any pixel can be changed (what matters is that it still resembles what is meant to resemble to a human), network features are constrained by protocols. This way, modifying a feature in the feature-space (e.g., reducing payload size) requires reconstructing a valid packet (in the traffic-space) that respects TCP/IP standards [6, 7]. This reconstruction step is often computationally intractable, making many attack strategies from other domains unfeasible in networking.

2.2.1 Brief Overview of Attacks

Attacks are categorized based on the adversary's knowledge and their goals [8].

Attacker Knowledge:

- **White-box:** The attacker has complete knowledge of the model (architecture, weights, parameters). They can use gradient-based optimization to craft precise attacks [3]—this consists of computing the gradient of the loss function with respect to the input features and adjusting them until the model misclassifies the input.
- **Black-box:** The attacker has no knowledge of the internal model. They can only query the system with inputs and observe the outputs (labels or label probability distribution). This is the most realistic scenario for NIDS, where attackers typically rely on transferability—the property that adversarial examples crafted for one model often fool other models too [27]. This is the primary threat model addressed in this work.
- **Gray-box:** The attacker has partial knowledge, such as the feature set used or the type of algorithm, but not the specific trained parameters.

Attacker Goals:

- **Poisoning:** Occurs during the *training phase*. The attacker injects malicious samples into the training data to compromise the model's integrity (creating backdoors) or availability (degrading overall accuracy in an indiscriminate fashion).
- **Evasion:** Occurs during the *inference phase*. The attacker modifies a malicious flow (e.g., by adding dummy packets or altering timing) to make it look benign to the classifier, allowing the attack to slip past the NIDS [9]. This is the primary threat addressed in this work.

2.2.2 Defenses

Defenses aim to harden models against these manipulations. Some of the most common techniques include:

- **Adversarial Training:** The model is trained on a mixture of clean and adversarial examples. While effective, it often reduces accuracy on benign traffic and is computationally expensive [28]. It's basically teaching the model to recognize the specific tricks an attacker might use, but it doesn't always generalize to new tricks.
- **Input Transformation:** This approach sanitizes inputs before classification. Generative models like GANs (represented in Figure 6) can be used to "purify" adversarial samples by reconstructing them onto the manifold of normal traffic. If the reconstruction error is high, or if the classification changes after reconstruction, the sample is flagged as adversarial [29, 30]. This effectively creates a "sanity check" for incoming data.

- **Defensive Distillation:** This technique uses a "teacher" network to train a "student" network using soft probabilities rather than hard labels. This smooths out the decision surface, making it much harder for attackers to find the sharp gradients they need to craft their attacks.
- **Certified Robustness:** These methods provide mathematical guarantees that a prediction will not change within a certain perturbation radius. However, they are currently difficult to scale to high-dimensional network data. That said, recent work like CertTA [31] has made huge strides here, providing certified robustness against multi-modal attacks (like changing both packet timing and size) which was previously a major hurdle.

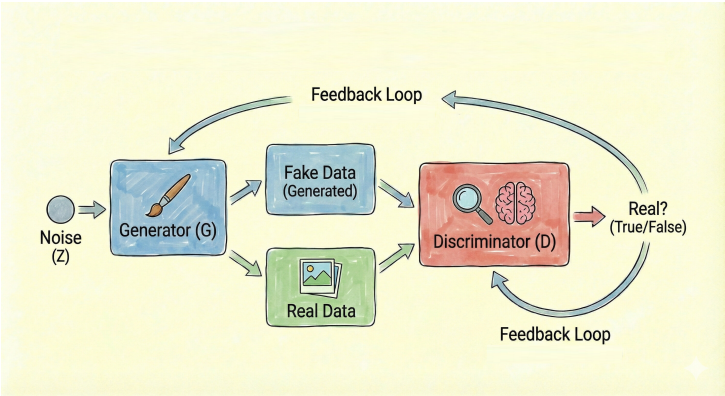


Figure 6: GAN Overview

2.3 ML-based Networks

The application of ML in networking extends beyond security, fueling the transition towards "self-driving" networks [2]. Below we explain key aspects of this landscape.

2.3.1 Granularity of Analysis

ML models in networking operate at different "granularities", and the choice between them determines what kind of attacks you can actually detect.

- **Packet-level:** Analyzes individual headers and payloads. It enables fast, real-time reactions but lacks historical context. This operates in the *traffic-space*, and often performs worse than the following analysis in terms of accuracy.
- **Flow-level:** Aggregates packets into sessions. It provides rich statistical context (e.g., jitter, throughput) suitable for detecting more complex behaviors but introduces latency as the system must typically wait for the flow to finish establishing. This often operates in the *feature-space*, utilizing derived metadata.

2.3.2 Generic ML Applications

ML is being integrated into various networking functions:

- **Traffic Engineering (TE):** Deep Reinforcement Learning (DRL) is used to optimize routing and load balancing dynamically, replacing static linear programming solvers [32, 33].
- **Congestion Control (CC):** Learning-based agents (like PCC Vivace [34] and Mutant [35]) adapt sending rates to real-time network conditions, outperforming fixed rules like *TCP Cubic*.

- **Foundation Models:** Large Language Models (LLMs), like NetLLM [36], are being adapted to interpret multimodal network logs and automate configuration tasks. Just like pre-training on text helps GPT understand English, pre-training on massive amounts of unlabeled network traffic allows these models to learn the "grammar" of protocols, which is useful when labeled attack data is scarce.

2.3.3 Network-Specific Challenges

Deploying ML in this domain faces unique constraints [6, 37]:

- **Latency and Throughput:** Modern networks operate at 100Gbps+. Inference must occur in microseconds to avoid introducing jitter.
- **Encryption:** Protocols like TLS and QUIC hide payloads, rendering Deep Packet Inspection (DPI) considerably less effective and forcing reliance on behavioral or less meaningful features.
- **Data Scarcity:** Labeled datasets are rare due to privacy concerns, leading to a "sim2real" gap where models trained on synthetic data fail when deployed in real networks.

2.4 Use of Ensemble Models

Ensemble learning improves performance by combining the predictions of multiple base models. The core principle is that a collective decision is often more accurate and robust than any single individual learner [10].

2.4.1 Ensemble Approaches

Bagging (Bootstrap Aggregating) trains homogeneous models (e.g., decision trees) in parallel on random data subsets to reduce variance (Figure 7) [10, 38]. The idea is simple: train multiple models on different "views" of the data and then average their predictions, which smooths out individual errors [39].

Boosting trains models sequentially, where each new model corrects the errors of the previous one, reducing bias (Figure 8) [40]. It's analogous to a student learning from their mistakes: each subsequent model focuses more on the hard-to-classify samples, gradually improving overall accuracy [10].

Stacking combines heterogeneous models (e.g., SVM + Neural Net + CNN) using a meta-learner (e.g., Transformer) to leverage their distinct strengths (Figure 9) [41]. This method captures complex patterns by learning how to best integrate diverse predictions. For this reason, stacking is particularly effective in adversarial settings, where diversity among base models can help mitigate transferable attacks [10, 42]. Furthermore, this meta-learner is perfect for our technical goal of analyzing architecture-level contributions to robustness, and for this, we decided to adopt it in our system.

2.4.2 Ensembles for Robustness

In the context of ADEPT, ensembles are particularly valuable for security. By decomposing the feature space into multiple low-dimensional AutoEncoders, **Kitsune** [15] increases the likelihood that malicious behavior manifests as an anomaly in at least one monitored subspace, improving robustness to subtle attacks. Furthermore, **Heterogeneous Stacking** mitigates attack transferability: an adversarial example optimized for a differentiable Neural Network often fails against a non-differentiable Random Forest [11], for example. This diversity creates a "rugged" decision boundary that is difficult for gradient-based attacks to bypass. Beyond just stopping attacks, ensembles are also great for handling *concept drift* (changes in data distribution over time)—systems like Mateen [43] use ensembles to adapt to changes in normal traffic patterns (benign shifts) by updating just parts of the ensemble rather than retraining everything, which keeps the defense fresh without expensive retraining.

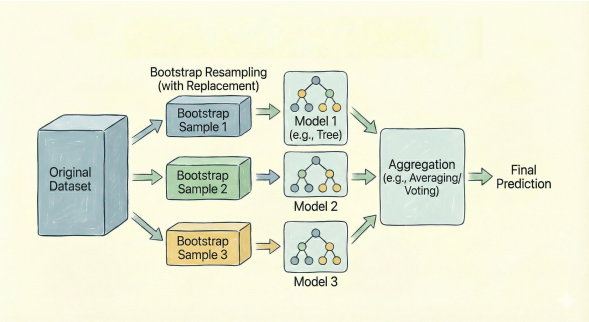


Figure 7: Bagging Ensemble Overview

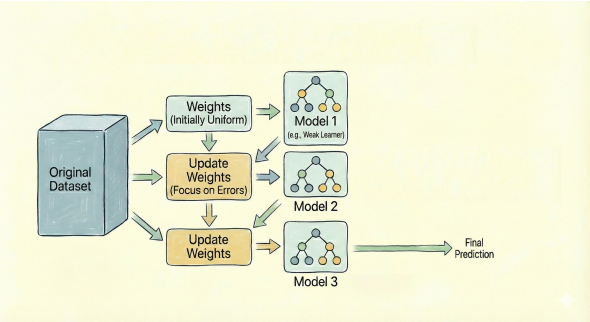


Figure 8: Boosting Ensemble Overview

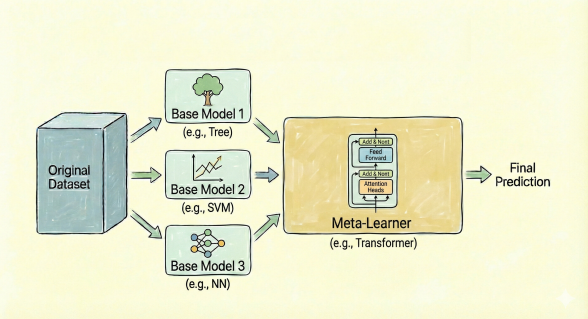


Figure 9: Stacking Ensemble Overview

3 Related Work

The escalating arms race between attackers and defenders in the domain of ML-based NIDS has catalyzed a significant body of research. While the previous section introduced the fundamental concepts, this section surveys the state-of-the-art in adversarial defenses. We categorize existing works into training-time strategies (hardening the model itself) and inference-time strategies (purifying the input before classification), followed by an analysis of Reinforcement Learning for adaptive defense. Finally, we identify gaps in the current literature that motivate our proposed approach.

3.1 Adversarial Defenses in ML-based NIDS

The following subsections review recent and prominent defense mechanisms against adversarial attacks in ML-based NIDS, segmented into training-time and inference-time strategies.

3.1.1 Training-time Defenses

Training-time defenses aim to construct models that are inherently resilient to manipulation before they are ever deployed. These strategies focus on sanitizing the training data or hardening the model's decision boundaries.

Statistical Sanitization and Poisoning Defense. Early research primarily focused on protecting the integrity of the training dataset itself. Rubinstein et al. [44] introduced **Antidote**, a mechanism that leverages robust statistics to identify and reject traffic flows that deviate significantly from the benign subspace. By filtering these outliers, they prevent attackers from injecting malicious biases during training. However, distinguishing poisoned samples from legitimate traffic bursts—which are common in multi-modal network data—remains a difficult challenge. To address this, recent approaches have turned to ensemble diversity. Biggio et al. [39] and Chen et al. [45] demonstrated that *Bagging* techniques offer a form of certified robustness against poisoning. The rationale is that since each base learner in the

ensemble sees only a random subset of the data, the impact of any specific malicious sample is diluted across the aggregate decision, making it harder for an attacker to corrupt the "global" model.

Federated and Split Learning Defenses. As NIDS architectures increasingly move towards distributed and Federated Learning (FL) to preserve privacy, defenses have evolved to inspect model updates (gradients) rather than the raw data itself. Wan et al. [46] proposed **FedTGE**, which utilizes Topological Graph Energy to detect backdoors in Federated Graph Learning. This method analyzes the structural energy of the graphs processed by clients, making it particularly relevant for analyzing complex network topology graphs where simple statistical metrics might fail. Complementing this, Xu et al. [47] introduced **AlignIns**, a defense that inspects the directional alignment of local model updates. However, statistical inspection of gradients can be evaded by adaptive attackers. To counter this, Rieger et al. [48] proposed **CrowdGuard**, which implements a secure client-feedback loop. Instead of the server passively aggregating updates, CrowdGuard distributes candidate models to clients who validate them against their local, private data. By analyzing the activation behavior of hidden layers—specifically looking for the high-magnitude paths characteristic of backdoors—and aggregating this feedback via a stacked clustering scheme inside Trusted Execution Environments (TEEs), CrowdGuard achieves 100% detection rates even in non-IID scenarios where client data heterogeneity typically blinds server-side defenses.

Backdoor Unlearning. When a model is found to be already compromised, retraining from scratch is often too costly. **SEAM** [49] proposes a method to induce "forgetting" of malicious correlations, effectively scrubbing backdoors from the weights while preserving benign accuracy. Similarly, **BayBFed** [50] applies Bayesian methods to identify and reject malicious gradients in Federated Learning, dynamically cleaning the model during the aggregation phase. Recent advancements have extended these unlearning concepts to Reinforcement Learning agents too. Gong et al. [51] proposed **TrajDeleter**, a framework for offline RL. It enables an agent to selectively "forget" specific hazardous trajectories—such as those containing poisoned attack traces—while maintaining performance on benign tasks. Nguyen et al. [52] introduced **PBP (Post-training Backdoor Purification)**. PBP effectively sanitizes compromised malware classifiers without requiring access to the original poisoned dataset or knowledge of the trigger. The method leverages the insight that "backdoor neurons" exhibit distinct activation distributions compared to benign neurons. By creating a binary mask of these hypersensitive neurons using a small set of clean data (approximately 1%) and applying masked gradient optimization to reverse their influence, PBP can reduce the Attack Success Rate (ASR) from 100% to near 0% while preserving benign accuracy. This allows for the remediation of deployed models without the cost of full retraining.

Realizable Adversarial Training. Finally, a major challenge in hardening network models is the aforementioned IFM problem: training on standard adversarial features is often useless if those features cannot exist in real packets. The following works, even though are attack-generator tools themselves, are useful for defense systems by integrating their generated examples into the training part of the defense framework. Addressing this, Jin et al. [6] introduced PANTS, a framework that integrates SMT solvers directly into the training loop. By mathematically verifying that every generated adversarial example is protocol-compliant, they ensure the model is hardened against realistic threats rather than impossible mathematical perturbations. However, while highly effective, the overhead of SMT solving limits this approach to offline hardening. The necessity of this strict realizability is further highlighted by DeepRed [53], a framework that uses GANs to generate command-and-control traffic. By ensuring strict adherence to TCP/IP constraints, DeepRed demonstrated that "realizable" attacks could degrade detection rates by up to 20%, proving that defenses must account for protocol validity during training. Following this line of reasoning, Pissarra [12] proposed TANTGen, a framework that shifts away from GANs and SMT solvers in favor of the Transformer architecture. By leveraging self-attention mechanisms, TANTGen is able to model the complex, sequential relationships between packet features, effectively generating adversarial traffic that is both protocol-compliant and capable of evading detection in black-box scenarios. Because it successfully addresses the difficulty of generating valid, realistic attacks without needing white-box access, this work served as the direct inspiration for the adversarial generation component of our solution.

3.1.2 Inference-time Defenses

Inference-time defenses focus on detecting evasion attacks during deployment, often by attempting to "purify" the input or by mathematically certifying the prediction stability.

Certified Robustness. Recent literature has moved towards providing mathematical guarantees for predictions. **CertTA** [31] adapts randomized smoothing to traffic analysis. This technique adds random noise to the flow features during inference and aggregates the predictions. If the consensus is strong enough, the system creates a "safe radius" around the flow where predictions are guaranteed to remain stable regardless of adversarial perturbation. However, standard randomized smoothing struggles with the discrete, sequential nature of network packets. To address this, Abad Rocamora et al. [54] introduced **LipsLev**, a framework providing deterministic robustness certificates under Bounded Levenshtein Distance. Unlike probabilistic smoothing which requires thousands of samples, LipsLev computes the Lipschitz constant of convolutional classifiers with respect to edit distance (insertions, deletions, and substitutions). By enforcing 1-Lipschitz continuity during training, it allows for the computation of a certified robustness radius in a single forward pass, making it four orders of magnitude faster than baseline verification methods and feasible for real-time traffic analysis.

Generative and Reconstruction-based Defenses. A core component of our proposed architecture is input purification, which aims to neutralize perturbations before they reach the classifier. **Defense-GAN** [29] originally proposed projecting adversarial inputs onto the benign manifold using GANs. The intuition is that an adversarial example lies just off the manifold of normal data—by finding the closest point on the manifold, the attack is removed. In the network domain, **AdvPurRec** [55] and **GTAE-IDS** [56] have adapted this concept using Diffusion Models and Transformer-based AutoEncoders, respectively. These models exploit self-attention mechanisms to reconstruct complex traffic sequences, effectively "denoising" the attack by regenerating the packet sequence based on learned benign patterns. Further advancing this domain, Li et al. [57] proposed **DiffuseDef**, which integrates a diffusion layer directly into the latent space of the classifier. Rather than diffusing raw inputs (which poses differentiability issues for discrete traffic tokens), DiffuseDef corrupts and then iteratively denoises the *hidden state representations* of the input sequence. By ensembling these purified latent states, DiffuseDef achieves state-of-the-art robustness against both white-box and black-box attacks, effectively pulling adversarial network flows back onto the manifold of benign semantics before classification occurs. Additionally, **CausalDiff** [58] integrates causal mechanisms to disentangle attack-specific factors from benign traffic features during reconstruction, ensuring that the purification process does not accidentally remove legitimate traffic characteristics. Similarly, Li et al. [59] proposed the **Adversarial Diffusion Bridge Model (ADBM)**, which constructs a direct reverse bridge from the adversarial distribution to the clean distribution to enhance purification fidelity.

Constraint-based Detection. Conversely, some defenses leverage the attacker's constraints against them. **NIDS-CBAD** [60] identifies adversarial attempts by flagging violations in inter-feature dependencies. For example, if an attacker naively perturbs the "payload size" feature without adjusting the "packet count" the inconsistency violates domain constraints. NIDS-CBAD flags these logical violations as adversarial, providing a lightweight defense that does not require complex generative models.

CNN-based Robustness and Vulnerability. While effective at feature extraction, CNN-based NIDS are inherently differentiable, rendering them susceptible to gradient-based evasion attacks such as the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD). Recent studies like **Tiki-Taka** [61] have demonstrated that attackers can exploit this differentiability to craft adversarial perturbations in time-based features, achieving evasion rates of up to 35.7% against state-of-the-art deep learning models. Consequently, current research emphasizes the need for heterogeneous ensembles that combine the feature-learning power of CNNs with non-differentiable models, or the integration of purification modules to sanitize inputs before they reach the convolutional layers. It is inspired by this study that our proposed architecture incorporates a purification stage and a non-differentiable classifier along with a CNN.

3.2 Reinforcement Learning for Continuous Adaptation

Reinforcement Learning (RL) is increasingly applied to make defenses dynamic, shifting from static models to adaptive agents. While **Mutant** [35] primarily uses online RL to adapt congestion control protocols, the concept of an agent that learns from the environment can be directly transferable to security policies as well. Newer frameworks have begun to address the complexities of automated response in multi-vector attack scenarios. **ARCS** [62] introduces a dual-stream RL architecture. One stream optimizes for immediate threat mitigation (reactive), while the second stream optimizes for long-term strategic defense (proactive). This separation allows the system to handle immediate attacks without losing sight of overall system health, showing superior performance against complex Advanced Persistent Threats (APTs). Beyond detection, recent work has focused on *autonomous response optimization*. Wilson et al. [63] introduced **Predictive Response Optimization (PRO)**. This system reframes defense as a Contextual Multi-Armed Bandit problem, where the agent selects from a spectrum of enforcement actions (e.g., CAPTCHA, friction, blocking) to optimize the trade-off between abuse reduction and user impact. In large-scale deployments, PRO reduced abuse by 59% by dynamically adapting its policy to shifting adversarial tactics. Simultaneously, for threat hunting, Qiao et al. [64] proposed **Slot**, which utilizes Graph Reinforcement Learning to navigate data provenance graphs. Slot treats the identification of APT attack paths as a graph traversal problem, learning to distinguish between causal attack chains and benign system noise, effectively automating the "hunting" phase of defense. Furthermore, **CAMP** [65] proposes a provably robust RL framework. It maximizes the "certified radius" of the agent's policy during training, ensuring that the defense remains stable even under worst-case perturbations of the network state. This provides a formal guarantee that the adaptive agent will not be easily tricked into a catastrophic action by a perturbed observation.

Despite these advancements, a gap remains: existing RL defenses focus on retraining policies or response actions rather than *adaptive input reconstruction*. To the best of our knowledge, there is currently no framework that combines the purification power of generative models with the continuous adaptability of RL to counter evolving threats in real-time. **ADEPT** aims to address this by integrating Transformer-based reconstruction with an RL feedback loop.

3.3 Summary

Table 1 summarizes the key characteristics of the surveyed state-of-the-art adversarial defenses in ML-based NIDS. While significant progress has been made in both training-time and inference-time defenses, several gaps remain. Training-time defenses often struggle with the IFM problem, as many adversarial examples used for hardening are not realizable in actual network traffic. Inference-time defenses, while effective at purification, often lack adaptability to evolving attack strategies. Moreover, the integration of RL for continuous adaptation remains underexplored, particularly in the context of input reconstruction. These gaps motivate our proposed approach, which seeks to combine the strengths of purification, ensemble learning, and adaptive RL to create a robust and dynamic defense mechanism against adversarial attacks in ML-based NIDS.

4 Solution

This section details the design of **ADEPT**, an adaptive framework engineered to protect Machine Learning-based Network Intrusion Detection Systems (NIDS). Our solution's layout is graphically summarized in Figure 10.

4.1 Threat Model

We operate under a realistic **black-box evasion** threat model. Unlike white-box scenarios where the attacker has access to the model's gradients, we assume the adversary can only query the system and observe the output labels. This mirrors real-world deployments where attackers treat NIDS as oracles to test their evasion strategies. Crucially, we also enforce a constraint of **functional realizability**. Many

Table 1: Comparison of State-of-the-Art Adversarial Defenses in Networking

Reference	Year	Strategy	Phase	Scope	Techniques
Rubinstein et al. [44]	2009	Stat. Sanitization	$\mathcal{T}$	Φ	Robust Statistics
B. Biggio et al. [39]	2011	Poisoning Def.	$\mathcal{T}$	Φ	Bagging Ensemble
P. Rieger et al. [48]	2024	FL Backdoor Def.	$\mathcal{T}$	Φ	TEE & Clustering
M. Jin et al. [6] (PANTS)	2025	Adv. Training	$\mathcal{T}$	Ψ	SMT Solvers
J. Yan et al. [31]	2025	Certified Robustness	$\mathcal{I}$	Ψ	Randomized Smoothing
P. Samangouei et al. [29]	2018	Purification	$\mathcal{I}$	Φ	GAN
Z. Li et al. [57]	2025	Purification	$\mathcal{I}$	Φ	Latent Diffusion
ADEPT	**2026**	**Purification + Ens.**	$\mathcal{I} + \mathcal{A}$	Ψ	**Stacking Ensemble + Transformer + RL**

Legend:
Phase: $\mathcal{T}$ Training-time, $\mathcal{I}$ Inference-time, $\mathcal{A}$ Continuous Adaptation.
Scope: Φ Feature-space (Abstract/Vector), Ψ Traffic-space (Realizable/Packet).
Techniques: SMT - Satisfiability Modulo Theories; FL - Federated Learning; TEE - Trusted Execution Env.; RL - Reinforcement Learning; GAN - Generative Adversarial Network.

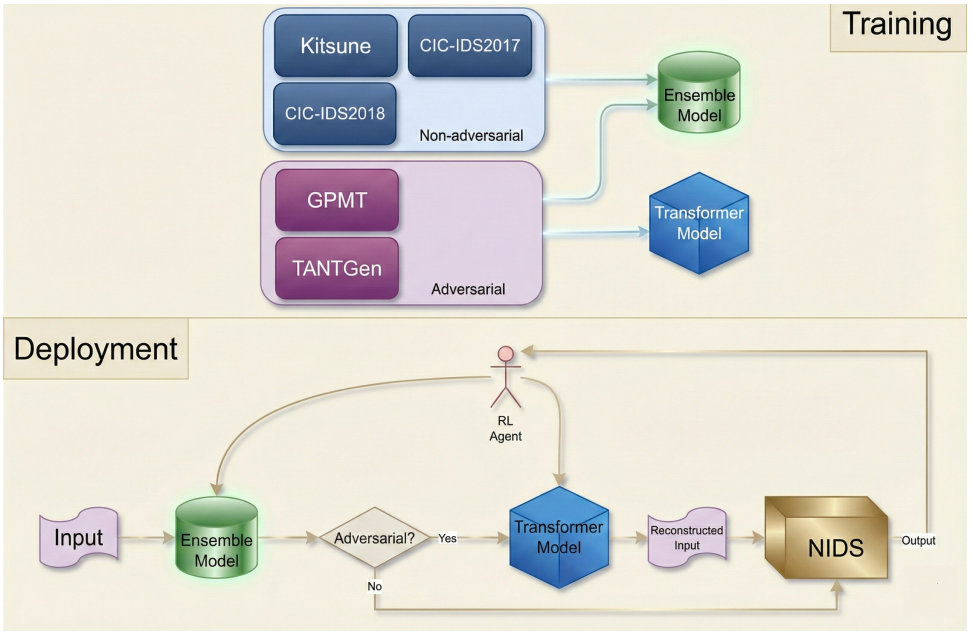


Figure 10: ADEPT Solution Overview

existing attacks manipulate features in a way that is mathematically powerful but protocol-invalid (e.g., negative inter-arrival times or inconsistent TCP checksums). Our defense assumes an attacker who, like the TANTGen framework, generates traffic that strictly adheres to protocol semantics. This way, if our defense cannot handle an invalid adversarial attack, we don't consider it a failure, since such an attack is unable to produce the desired malicious outcome.

4.2 ADEPT Architecture Overview

To make matters clearer, we define the following mentioned concepts as: 1) normal traffic: network packets/flows that have not been adversarially manipulated (can be either benign or malicious—e.g a SYN Flood); 2) adversarial traffic: network data that has been adversarially tampered with. As we can see in Figure 10, the system functions as a continuous, closed-loop feedback system rather than a static filter. The first step is **Training**, where we train the ensemble detector—the objective is to create a robust ensemble that can identify a wide variety of attack vectors, including those specifically designed to evade detection—and the purification module, ensuring it focuses on learning to project adversarial inputs back onto the manifold of normal traffic. The main deployment workflow then begins with 1) incoming traffic is first inspected by our **Primary Detection** module, an ensemble of supervised models; 2) if a packet is flagged as adversarial, it is routed to the **Input Purification** module; 3) here, a Transformer-based model attempts to "denoise" the packet, projecting it back onto the learned manifold of normal traffic; 4) the purified packet is then routed to the NIDS for processing; 5) finally, a **Reinforcement Learning** agent monitors the performance of both the ensemble and purification modules, adjusting their parameters in real-time to adapt to evolving attack strategies.

4.2.1 Rationale for Model Selection

The architectural choices for ADEPT are driven by specific failures observed in single-model defenses and the necessity to counter powerful evasion strategies such as gradient-based perturbation. We adopt a stacking approach that leverages the complementary strengths of heterogeneous algorithms.

Heterogeneous Ensemble Stacking for Gradient Masking We employ a heterogeneous ensemble combining a **Random Forest (RF)** and a **1D Convolutional Neural Network (1D-CNN)**. This choice is strategic and based in the principle of gradient masking. Neural Networks, while powerful feature extractors, are differentiable functions. This property makes them inherently susceptible to white-box and query-based attacks (e.g., PGD, FGSM) where an attacker calculates the precise perturbation δ required to minimize the loss function with respect to a target class. Conversely, Random Forests, being composed of discrete decision trees, create non-differentiable decision boundaries. By stacking these two architectures, we create a "rugged" decision surface. An adversarial example optimized to fool the smooth gradient of the 1D-CNN will likely fail against the discrete thresholds of the Random Forest, significantly reducing the transferability of attacks [11, 66]—this is the reason why we say our system focuses on defending against black-box and not white-box capabilities. The inclusion of the 1D-CNN is critical for extracting local spatial features from packet sequences, a capability where tree-based models often underperform compared to deep learning architectures [67].

Attention-Based Meta-Learning (the "Mini-Transformer") To aggregate the predictions of the base learners, we opted to reject traditional linear meta-learners (e.g., Logistic Regression) in favor of a (small) **Transformer** architecture, functioning as an Attention-Based Meta-Learner. In adversarial settings, base learners often exhibit "overconfidence" on misclassified samples—a phenomenon known as calibration error. Standard voting or linear weighting schemes fail to detect this nuance and may be swayed by a compromised model's high confidence. Recent research demonstrates that Transformer-based meta-learners can effectively capture the non-linear correlations between base model predictions [42]. By utilizing a self-attention mechanism, our meta-learner dynamically assigns weights to the RF and 1D-CNN outputs conditioned on the specific input. This effectively acts as a dynamic gating mechanism: if

the 1D-CNN exhibits activation patterns characteristic of gradient-based manipulation or disagrees fundamentally with the robust Random Forest, the attention mechanism learns to suppress its contribution. This allows ADEPT to not just simply average decisions, but to intelligently select the most reliable expert for a given traffic flow.

Decoder-Only Transformers for Real-Time Purification For the purification module, we select a **decoder-only Transformer** architecture (similar to GPT—Figure 11) rather than a standard AutoEncoder or GAN. While GANs are excellent for image generation, they often struggle with the sequential dependencies inherent in network flows (e.g., the causal relationship between a TCP SYN and an ACK). We specifically favor the decoder-only architecture over encoder-decoder models (like BERT or T5) for efficiency. In a high-speed network environment (100Gbps+), latency is crucial. Decoder-only models allow us to utilize Key-Value (KV) caching, which significantly reduces the computational cost of generating token sequences (packets) step-by-step. This ensures that our "denoising" process does not introduce unacceptable network latency.

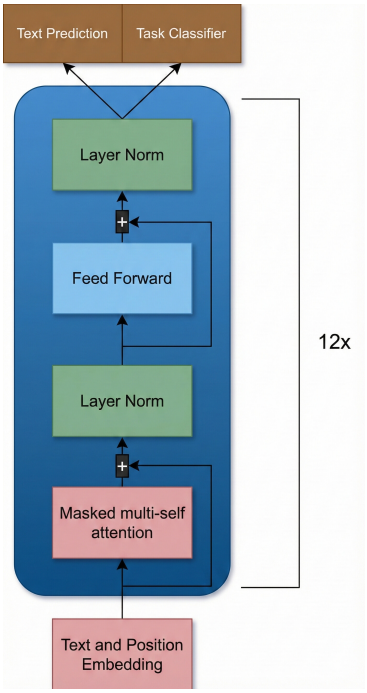


Figure 11: Decoder-Only Transformer Architecture (GPT-1)

Reinforcement Learning for Continuous Adaptation Static defenses historically fail because they can be adapted to—once an attacker learns the fixed rules, they bypass them. We integrate an RL agent to turn the defense into a moving target. The RL agent treats the detection accuracy as a reward signal, utilizing feedback to adjust the system's decision boundaries in two key ways [18]:

1. **Dynamic Threshold Adjustment:** The agent learns to shift the classification confidence threshold τ in real-time. Basically, if the system detects a high rate of false alarms, the agent raises the threshold to require higher confidence for flagging; conversely, during an active evasion campaign, it lowers the threshold to catch subtle perturbations.
2. **Ensemble Weight Adaptation:** The agent dynamically updates the contribution weights of the base learners (RF and 1D-CNN) within the meta-learner. This ensures that if one architecture is being specifically targeted by a gradient-based attack, its influence on the final decision is automatically reduced.

This continuous feedback loop ensures that both modules evolve in response to emerging threats, effectively making the decision boundary "adaptive" rather than static.

4.2.2 Dataset Creation

A major hurdle in ML-NIDS is the "sim2real" gap, where models trained on synthetic data fail in production. To mitigate this, we combine the high-quality, human-labeled attack scenarios from public benchmarks like **CIC-IDS2018** [68] with raw, "normal" traffic streams from **Kitsune** [69] to capture the noise of real networks. Furthermore, we augment this dataset with "hard" examples generated by GPMT and TANTGen. Training our model on these diverse, protocol-compliant adversarial samples teaches it to recognize the subtle perturbations that distinguish a "realizable" attack from benign noise.

4.2.3 Preprocessing and Feature Engineering

Since Transformers are natively designed for discrete data (text), applying them to continuous network metrics (like "Flow Duration") requires careful adaptation. We adopt the discretization approach proposed in **NetFlowGen** [70]. Continuous features are binned into discrete tokens (e.g., "Duration: Short", "Duration: Medium").

This discretization serves two purposes:

1. **Compatibility:** It aligns the network data with the categorical nature of the Transformer's cross-entropy loss function.
2. **Robustness:** It acts as a low-pass filter. Small adversarial perturbations to a continuous value (e.g., changing a duration from 0.05s to 0.051s) often fall into the same "bin", effectively neutralizing micro-perturbation attacks before they even reach the model.

Additionally, in regards to the Ensemble module, we perform feature normalization and selection to ensure that both the Random Forest and Neural Network operate on a consistent feature space, enhancing their complementary strengths.

4.2.4 Defense Application Loop in Real-life

The final application we intend operates as an autonomous agent. When the ensemble flags a packet, the Transformer reconstructs it. The RL agent monitors this process: later, if the packet is found to have been misclassified or incorrectly purified, it adjusts the parameters of the ensemble and the Transformer, respectively, to improve future performance.

4.3 Tools and Frameworks

Implementation relies on an open-source stack:

- **PyTorch & HuggingFace:** Chosen for their modularity in building custom Transformer architectures.
- **Stable Baselines3:** Utilized for implementing the Reinforcement Learning agent, providing a suite of pre-built algorithms and training utilities.
- **Scapy & CICFlowMeter:** Essential for parsing the raw packet data and extracting flow-level features that respect the protocol constraints discussed in Section 4.1.

5 Evaluation

In this section, we outline the experimental strategy to validate ADEPT. The goal is not just to prove that the system detects attacks, but to demonstrate that it can actively adapt to and neutralize evasion attempts that would otherwise bypass standard NIDS. We detail the research questions that guide our experiments, the setup used to simulate realistic network threats, and the specific metrics we will use to measure success.

5.1 Research Questions

We aim to address the following research questions to evaluate the robustness and practicality of our solution:

- **Ensemble Robustness:** Does the heterogeneous ensemble (Random Forest + 1D-CNN) actually limit the transferability of gradient-based attacks compared to single-architecture models? We need to verify if the "rugged" decision boundary theory holds in practice.
- **Purification Effectiveness:** Can the Transformer-based purification module successfully project adversarial inputs back onto the benign manifold without destroying the packet's validity? It is crucial that the "denoised" packet remains protocol-compliant.
- **RL Adaptability:** Does the Reinforcement Learning agent successfully optimize the defense policy over time? We want to see if the system effectively "learns" from the feedback.
- **Latency and Overhead:** Can ADEPT operate within the strict latency and throughput requirements of high-speed networks? Since we are introducing a Transformer in the critical path (for suspicious packets), we must ensure that the introduced overhead does not result in unacceptable latency, jitter, packet loss, or throughput degradation.

5.2 Experimental Setup

The experiments will be conducted in a controlled environment designed to simulate a real-world enterprise network facing active evasion attempts. The setup consists of three main components: the Traffic Generation tool (TANTGen), the ADEPT Defense System (our solution), and the Target NIDS.

5.2.1 Datasets and Threat Generation

To ensure our results are comparable with state-of-the-art literature, we utilize the following datasets:

- **Normal attacks:** We will use **CIC-IDS2017** [71], **CSE-CIC-IDS2018** [68], and **Kitsune** [69] to represent standard background traffic and known attack patterns (e.g., DDoS, Brute Force, Botnet). This ensures our base ensemble is grounded in realistic traffic data.
- **Adversarial Generation (The Attacker):** To test the defense against sophisticated evasion, we will employ **TANTGen**. This tool will act as the "Red Team", generating protocol-compliant adversarial packets specifically designed to fool our ensemble. This allows us to test the system against fresh, realistic black-box attacks rather than just theoretical perturbations.

5.2.2 Implementation Environment

The defense framework is implemented using **PyTorch** for the neural network components (1D-CNN and Transformer) and **Scikit-learn** for the Random Forest elements.

- **Reinforcement Learning:** We utilize **Stable Baselines3** to implement the RL agent.
- **Packet Handling:** **Scapy** and **CICFlowMeter** are used for parsing raw PCAP files and extracting flow features in real-time.

- **Hardware:** The experiments are conducted on a server equipped with two Intel Xeon Gold 5320 CPUs (52 cores), 192GB of RAM, and four NVIDIA RTX A4000 GPUs.

5.3 Metrics

To provide a comprehensive assessment, we evaluate ADEPT using three distinct groups of metrics:

1. Classification and Robustness

- **Clean Accuracy & F1-Score:** The system's performance on standard, non-adversarial traffic. We must ensure that adding the defense does not degrade performance on normal data.
- **Attack Success Rate (ASR):** The percentage of adversarial samples generated by TANTGen that successfully bypass the NIDS. A lower ASR indicates a more robust defense.

2. Reconstruction Quality

- **Purification Gain:** The difference in classification confidence between the original adversarial packet and the purified packet. This can be achieved by feeding the reconstructed packet back into the ensemble and registering the change in the prediction.
- **Protocol Compliance Rate:** The percentage of purified packets that remain valid according to the TCP/IP stack. If the Transformer "fixes" the attack but breaks the protocol (e.g., invalid checksums or flags), the packet is useless.

3. Efficiency and Performance

- **Throughput (pps):** The number of packets per second the system can process.
- **Inference Latency (ms):** The time taken to process a single flow. We will specifically measure attentively the "Purification Overhead"—the extra time added when the Transformer is triggered.
- **Resource Utilization:** GPU memory usage and CPU load during testing run.

6 Work Schedule

The table in Figure 12 describes the proposed schedule for the development of ADEPT, divided into specific tasks and milestones. In summary, by late August we intend to have the system fully implemented, including the ensemble training and the reinforcement learning automation. From then on we want to focus on the experimental evaluation to validate the defense mechanisms, which should be completed by early October. Finally, the writing process will continue in parallel, aiming for the delivery of the final dissertation in late October.

7 Conclusion

The rapid integration of Machine Learning into network security has created a new attack surface: adversarial evasion. While recent research has focused heavily on generating these attacks, the development of practical defenses that can operate under realistic network constraints remains a significant challenge. Most existing defenses either assume unrealistic attacker knowledge or fail to account for the strict protocol validity required in network traffic. This thesis proposes ADEPT, an adaptive defense framework designed to bridge the gap between theoretical robustness and real-world deployability. By combining a heterogeneous ensemble of Random Forests and 1D-CNNs, coupled with the rising popularity of the Transformer as the Meta-Learner, we create a model that is inherently resistant to gradient-based perturbation transferability. Furthermore, we introduce a Transformer-based purification module that leverages the sequential nature of network flows to project adversarial inputs back onto the manifold of normal traffic, effectively "denoising" attacks in real time while maintaining protocol compliance.

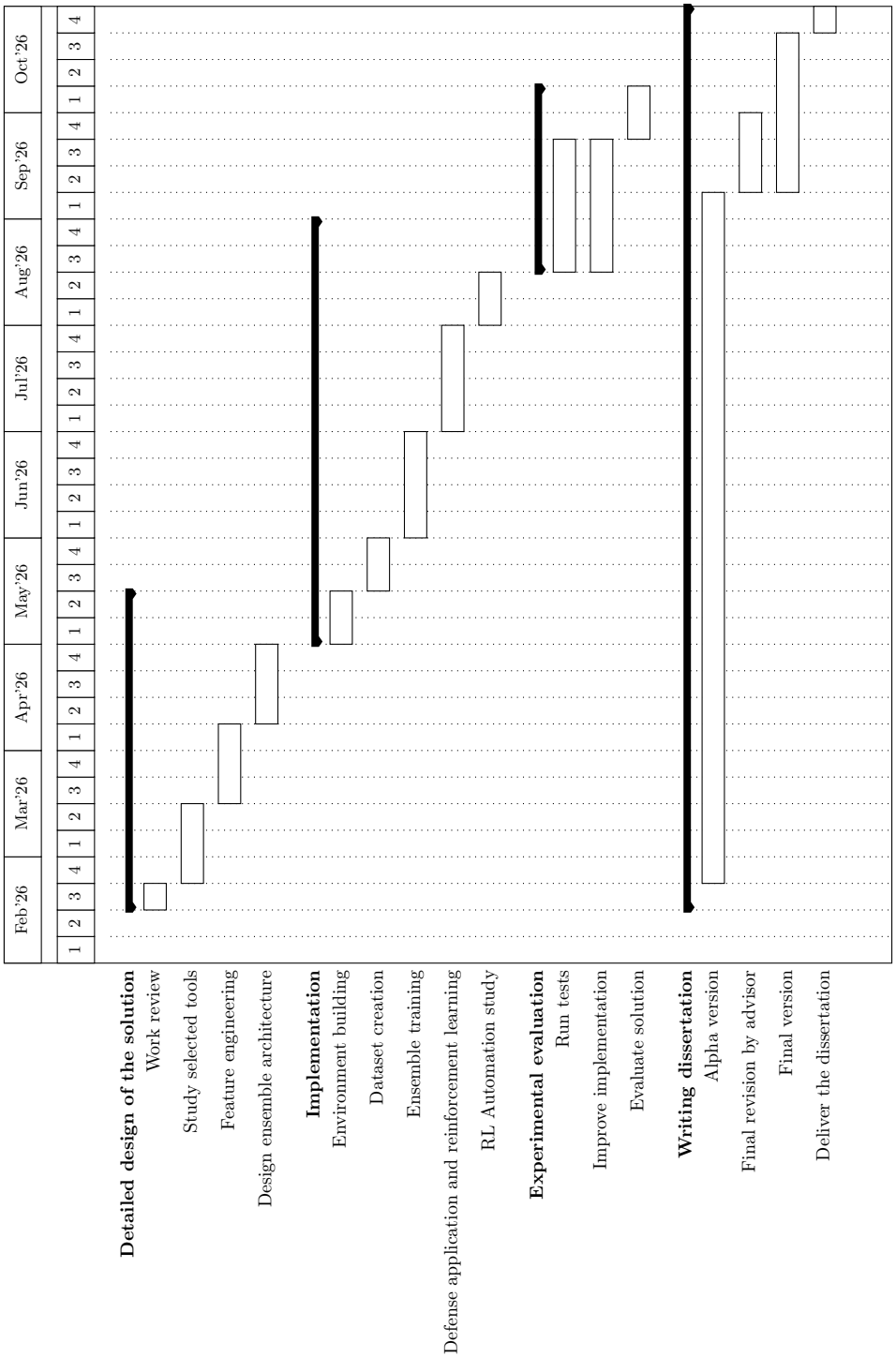


Figure 12: Planned Schedule

Bibliography

[1] S. Ennaji, F. De Gaspari, D. Hitaj, A. Kbidi, and L. V. Mancini, "Adversarial challenges in network intrusion detection systems: Research insights and future prospects," *IEEE Access*, 2025.

[2] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, pp. 1–20, 2018.

[3] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *2nd International Conference on Learning Representations (ICLR 2014)*, 2014.

[4] M. J. Hashemi, G. Cusack, and E. Keller, "Towards evaluation of nidss in adversarial setting," in *Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks*, 2019, pp. 14–21.

[5] P. Sun, S. Li, J. Xie, H. Xu, Z. Cheng, and R. Yang, "Gpmt: Generating practical malicious traffic based on adversarial attacks with little prior knowledge," *Computers & Security*, vol. 130, p. 103263, 2023.

[6] M. Jin and M. Apostolaki, "Robustifying {ML-powered} network classifiers with {PANTS}," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 7291–7310.

[7] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing properties of adversarial ml attacks in the problem space," in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1332–1349.

[8] K. He, D. D. Kim, and M. R. Asghar, "Adversarial machine learning for network intrusion detection systems: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 538–566, 2023.

[9] G. Apruzzese, P. Laskov, and J. Schneider, "Sok: Pragmatic assessment of machine learning for network intrusion detection," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 592–614.

[10] T. J. Lucas, I. S. De Figueiredo, C. A. C. Tojeiro, A. M. G. De Almeida, R. Scherer, J. R. F. Brega, J. P. Papa, and K. A. P. Da Costa, "A comprehensive survey on ensemble learning-based intrusion detection approaches in computer networks," *IEEE Access*, vol. 11, pp. 122 638–122 676, 2023.

[11] Z. Yang, L. Li, X. Xu, S. Zuo, H. Qian, and J. Rubin, "Trs: Transferability reduced ensemble via promoting gradient diversity and model smoothness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[12] I. d. A. Pissarra, "Tantgen: Transformer-based adversarial network traffic generator," Master's thesis, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal, 2026, unpublished Master's Thesis.

[13] S. Wang, J. F. Balarezo, S. Kandeepan, A. Al-Hourani, K. G. Chavez, and B. Rubinstein, "Machine learning in network anomaly detection: A survey," *IEEe Access*, vol. 9, pp. 152 379–152 396, 2021.

[14] Z. Xu, Y. Wu, S. Wang, J. Gao, T. Qiu, Z. Wang, H. Wan, and X. Zhao, "Deep learning-based intrusion detection systems: A survey," *arXiv preprint arXiv:2504.07839*, 2025.

[15] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Network and Distributed System Security Symposium (NDSS)*, 2018.

[16] I. H. Sarker, "Deep cybersecurity: a comprehensive overview from neural network and deep learning perspective," *SN Computer Science*, vol. 2, no. 3, p. 154, 2021.

[17] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2021.

[18] G. Caminero, M. Lopez-Martin, and B. Carro, "Adversarial environment reinforcement learning algorithm for intrusion detection," in *Computer Networks*, vol. 159, 2019, pp. 96–109.

[19] A. Salim, O. Salim, O. M. Khudhur, S. M. Al-Barzinji, and F. M. Jasem, "Feature selection techniques in intrusion detection systems: A review," *Journal of Cybersecurity and Information Management (JCIM) Vol*, vol. 17, no. 02, pp. 97–112, 2026.

[20] S. Ning, K. Nguyen, S. Bagchi, and Y. Park, "The study of feature engineering in machine learning and deep learning for network intrusion detection systems," in *2024 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 2024, pp. 1–5.

[21] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[22] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *2017 IEEE international conference on intelligence and security informatics (ISI)*. IEEE, 2017, pp. 43–48.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[24] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.

[25] F. M. Anis, M. AlAbdullatif, S. Aljbli, and M. Hammoudeh, "A survey on the applications of deep learning in network intrusion detection systems to enhance network security," *IEEE Access*, 2025.

[26] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[27] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, 2017, pp. 506–519.

[28] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *6th International Conference on Learning Representations (ICLR)*, 2018.

[29] P. Samangouei, M. Kabkab, and R. Chellappa, "Defense-GAN: Protecting classifiers against adversarial attacks using generative models," in *International Conference on Learning Representations (ICLR)*, 2018.

[30] F. Zuo, Y. Li, P. Young, L. Nelson, Y. Zhang, and K. Zeng, "Exploiting the inherent limitation of l0 adversarial examples," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 293–310.

[31] J. Yan, Z. Li, Y. Zhang, Y. Chen, and X. Wang, "Certta: Certified robustness made practical for learning-based traffic analysis," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025.

[32] L. Chen, J. Lingys, K. Chen, and F. Liu, "Auto: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*. ACM, 2018, pp. 191–205.

[33] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, "Teal: Learning-accelerated optimization of wan traffic engineering," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 378–393.

[34] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, "Pcc vivace: Online-learning congestion control," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, 2018, pp. 343–356.

[35] L. Pappone, A. Sacco, and F. Esposito, "Mutant: Learning congestion control from existing protocols via online reinforcement learning," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 2025, pp. 1507–1522.

[36] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "Netllm: Adapting large language models for networking," in *Proceedings of the 2024 ACM SIGCOMM 2024 Conference (SIGCOMM '24)*. ACM, 2024.

[37] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "In-network machine learning using programmable network devices: A survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1171–1200, 2023.

[38] L. Breiman, "Bagging predictors," *Machine learning*, vol. 24, pp. 123–140, 1996.

[39] B. Biggio, I. Corona, G. Fumera, G. Giacinto, and F. Roli, "Bagging classifiers for fighting poisoning attacks in adversarial classification tasks," in *Multiple Classifier Systems: 10th International Workshop (MCS 2011)*. Springer, 2011, pp. 350–359.

[40] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of computer and system sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[41] D. H. Wolpert, "Stacked generalization," *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.

[42] X. Yang, Y. Zhao, and X. Chen, "A novel transformer-based stacking ensemble method with multi-model integration for cancer classification," *PeerJ Computer Science*, vol. 11, p. e3314, 2025.

[43] F. Alotaibi and S. Maffeis, "Mateen: Adaptive ensemble learning for network anomaly detection," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, 2024, pp. 215–234.

[44] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S.-h. Lau, S. Rao, N. Taft, and J. D. Tygar, "Antidote: understanding and defending against poisoning of anomaly detectors," in *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*, 2009, pp. 1–14.

[45] R. Chen, Z. Li, J. Li, J. Yan, and C. Wu, "On collective robustness of bagging against data poisoning," in *International Conference on Machine Learning*. PMLR, 2022, pp. 3299–3319.

[46] G. Wan, Z. Shi, W. Huang, G. Zhang, D. Tao, and M. Ye, "Energy-based backdoor defense against federated graph learning," in *The Thirteenth International Conference on Learning Representations*, 2025.

[47] J. Xu, Z. Zhang, and R. Hu, "Detecting backdoor attacks in federated learning via direction alignment inspection," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 20 654–20 664.

[48] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A.-R. Sadeghi, "Crowdguard: Federated backdoor detection in federated learning," in *Network and Distributed System Security Symposium (NDSS)*, 2024.

[49] R. Zhu, D. Tang, S. Tang, X. Wang, and H. Tang, "Selective amnesia: On efficient, high-fidelity and blind suppression of backdoor effects in trojaned machine learning models," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1–19.

[50] K. Kumari, P. Rieger, H. Fereidooni, M. Jadliwala, and A.-R. Sadeghi, "Baybfed: Bayesian backdoor defense for federated learning," in *2023 IEEE symposium on security and privacy (SP)*. IEEE, 2023, pp. 737–754.

[51] C. Gong, K. Li, J. Yao, and T. Wang, "Trajdeleter: Enabling trajectory forgetting in offline reinforcement learning agents," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[52] D. T. Nguyen, N. N. Tran, T. T. Johnson, and K. Leach, "Pbp: Post-training backdoor purification for malware classifiers," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[53] M. Hajizadeh, P. Golchin, E. Nowroozi, M. Rigaki, V. Valeros, S. García, M. Conti, and T. Bauschert, "{DeepRed}: A deep {Learning–Powered} command and control framework for {Multi-Stage} red teaming against {ML-based} network intrusion detection systems," in *19th USENIX WOOT Conference on Offensive Technologies (WOOT 25)*, 2025, pp. 103–127.

[54] E. Abad Rocamora, G. G. Chrysos, and V. Cevher, "Certified robustness under bounded levenshtein distance," in *International Conference on Learning Representations (ICLR)*, 2025.

[55] N. Alhussien and A. Aleroud, "Advpurrec: Strengthening network intrusion detection with diffusion model reconstruction against adversarial attacks," in *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2024, pp. 1638–1646.

[56] J. Ghadermazi, S. Hore, A. Shah, and N. D. Bastian, "Gtae-ids: Graph transformer-based autoencoder framework for real-time network intrusion detection," *IEEE Transactions on Information Forensics and Security*, 2025.

[57] Z. Li, H. Zhou, M. Rei, and L. Specia, "Diffusedef: Improved robustness to adversarial attacks via iterative denoising," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 9259–9274.

[58] M. Zhang, K. Bi, W. Chen, Q. Chen, J. Guo, and X. Cheng, "Causaldiff: Causality-inspired disentanglement via diffusion model for adversarial defense," *Advances in Neural Information Processing Systems*, vol. 37, pp. 124 791–124 819, 2024.

[59] X. Li, W. Sun, H. Chen *et al.*, "Adbm: Adversarial diffusion bridge model for reliable adversarial purification," in *International Conference on Learning Representations (ICLR)*, 2025.

[60] V. Kumar, K. Kumar, and M. Singh, "Nids-cbad: detecting adversarial attacks in network intrusion detection systems using domain constraints," *International Journal of Machine Learning and Cybernetics*, vol. 16, no. 7, pp. 4213–4234, 2025.

[61] C. Zhang, X. Costa-Pérez, and P. Patras, "Tiki-taka: Attacking and defending deep learning-based intrusion detection systems," in *Proceedings of the 2020 ACM SIGSAC conference on cloud computing security workshop*, 2020, pp. 27–39.

[62] S. Ren, J. Jin, G. Niu, and Y. Liu, "Arcs: Adaptive reinforcement learning framework for automated cybersecurity incident response strategy optimization," *Applied Sciences*, vol. 15, no. 2, p. 951, 2025.

[63] G. Wilson, G. Goh, Y. Jiang, A. Gupta, J. Wang, D. Freeman, and F. Dinuzzo, "Predictive response optimization: Using reinforcement learning to fight online social network abuse," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[64] W. Qiao, Y. Feng, T. Li, Z. Ma, Y. Shen, J. Ma, and Y. Liu, "Slot: Provenance-driven apt detection through graph reinforcement learning," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 963–977.

[65] D. Wang *et al.*, "Camp in the odyssey: Provably robust reinforcement learning with certified radius maximization," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[66] O. Gungor, T. Rosing, and B. Aksanli, "Stewart: Stacking ensemble for white-box adversarial attacks towards more resilient data-driven predictive maintenance," *Computers in Industry*, vol. 140, p. 103660, 2022.

[67] I. Bibers, O. Arreche, W. Alayed, and M. Abdallah, "Ensemble-ids: An ensemble learning framework for enhancing ai-based network intrusion detection tasks," *Applied Sciences*, vol. 15, no. 19, p. 10579, 2025.

[68] "A realistic cyber defense dataset (CSE-CIC-IDS2018) - registry of open data on AWS," [Online]. Available: https://registry.opendata.aws/cse-cic-ids2018/, accessed: 2025-12-30.

[69] "Kitsune Network Attack," UCI Machine Learning Repository, 2019, DOI: https://doi.org/10.24432/C5D90Q.

[70] J. Zhou, W. Kim, Z. Xu, A. M. Rush, and M. Yu, "Netflowgen: Leveraging generative pre-training for network traffic dynamics," *arXiv preprint arXiv:2412.20635*, 2024.

[71] "Intrusion detection evaluation dataset (CIC-IDS2017)," [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html, accessed: 2025-12-30.